Pipeline for metagenomic sequence data processing:

Sequence QC:

FASTQ sequences were pre-processed with KneadData[1] (v0.12.3) prior to taxonomic classification. The steps in KneadData are:

1) Use Trimmomatic[2] to trim Illumina adapters, low-quality bases (using a sliding window of 4, and clipping the read if the average window Phred score drops below 20), and filter resulting reads if their length is <50% of the input read length

2) Remove repetitive sequences using Tandem Repeats Finder (TRF)[3]

3) Remove reads from human DNA using bowtie2[4], searching against a database of human DNA (T2T-CHM13v2.0)

For each sample ($SAMPLE), KneadData was run on the paired-end fastq files, using a bowtie2 database built from the T2T-CHM13v2.0 human reference genome. Additional options are used to keep the read pairs in the same order as the original input fastq files (option: --reorder), and otherwise was run with default settings:

```
kneaddata --input1 $DATA_DIR/${SAMPLE}_*R1*.fastq.gz \
      --input2 $DATA_DIR/${SAMPLE}_*R2*.fastq.gz \
      --reference-db $DB_DIR \
      --output $INT_OUTPUT_DIR \
      --threads $NSLOTS \
      --reorder \
      --output-prefix ${SAMPLE}_kneaddata
```

KneadData outputs 4 processed and filtered fastq file output and log output:

1. $SAMPLE_kneaddata_paired_1.fastq.gz — Reads from the first paired-end file that pass all KneadData filters *and whose mate also passes*.
2. $SAMPLE_kneaddata_paired_2.fastq.gz — Reads from the second paired-end file that pass all filters *and whose mate also passes*.
3. $SAMPLE_kneaddata_unmatched_1.fastq.gz — Reads from the first paired-end file that pass filtering *even though their mate does not*.
4. $SAMPLE_kneaddata_unmatched_2.fastq.gz — Reads from the second paired-end file that pass filtering *even though their mate does not*.
5. $SAMPLE_kneaddata.log — Log file summarizing run statistics and filtering outcomes.

A summary table is then generated of the total number of reads kept at each step of the Kneaddata process for each sample:

```
kneaddata_read_count_table --input $INT_OUTPUT_DIR  \
      --output $FINAL_OUTPUT_DIR/${SAMPLE}_kneaddata_read_counts.txt
```

FASTQC reports are generated after pre-processing with KneadData to assess the sequence quality of samples. MultiQC is used to combine the FastQC reports across all samples into a single report[5].

K-mer-based taxonomic classification: Kraken2[6,7] is a tool for taxonomic classification of shotgun metagenomic sequencing data using a k-mer-based approach. The classifier assigns taxonomic labels to each sequencing read by querying k-mer minimizer sequences within a read against a database. A custom database is built from complete genomes in RefSeq for the bacterial, archaeal, viral, and plasmid domains, along with the human T2T-CHM13v2.0 genome, and collections of eukaryotic pathogens (EuPathDB) and known vectors (UniVec_Core) (downloaded and built June 25, 2025).

1. Download taxonomy

```
kraken2-build --download-taxonomy --db $DBNAME
```

2. Download RefSeq complete genome libraries

```
kraken2-build --threads 28 --download-library archaea --db $DBNAME
kraken2-build --threads 28 --download-library bacteria --db $DBNAME
kraken2-build --threads 28 --download-library plasmid --db $DBNAME
kraken2-build --threads 28 --download-library viral --db $DBNAME
kraken2-build --threads 28 --download-library human --db $DBNAME
kraken2-build --threads 28 --download-library UniVec_Core --db $DBNAME
```

3. Download T2T-CHM13v2.0 human reference sequence to be added to human reference

```
kraken2-build --add-to-library ${DB_DIR}/chm13v2.0.fa --db $DBNAME
```

4. Download EuPathDB sequences

```
curl -O -s http://ccb.jhu.edu/data/eupathDB/dl/eupathDB.tar.gz
```

5. Build database based on libraries

```
kraken2-build --threads 28 --build --db $DBNAME
```

Kraken2 v. 2.1.3 is run on each sample with paired-end fastq input files from KneadData output ($SAMPLE_R1_001_kneaddata.fastq.gz) and with the Kraken2Uniq option to report the k-mer minimizer counts (--report_minimizer_data). Additional options are used to set a minimum number of overlapping k-mers sharing the same minimizer (--minimum-hit-groups) and report all taxa including those that have zero read counts (--report-zero-counts):

```
kraken2 --threads 16 --db $DBNAME \
        --output $SAMPLE.kraken.txt \
        --report $SAMPLE.aggregated.report.txt \
        --minimum-hit-groups 4 \
        --report-minimizer-data \
        --report-zero-counts \
        --use-names \
        --paired \
        --gzip-compressed $SAMPLE__kneaddata_paired_1.fastq.gz
      $SAMPLE__kneaddata_paired_2.fastq.gz
```

Kraken2 with the Kraken2Uniq option provides two outputs:
1) Standard output file ($SAMPLE.kraken.txt) which indicates how each input sequence pair was classified including the taxa that each k-mer mapped back to, to determine the final taxon label.
2) Report file ($SAMPLE.aggregated.report.txt) which identifies the taxa classified for the read sequences and the number of reads and k-mer minimizer sequences mapped to each taxon.

From the Kraken2 report files, we use a custom R script (02_kraken2uniq-generate-table.R) to merge the k-mer minimizer counts across samples into a table of k-mer minimizer counts with samples as columns and taxa as rows to use for downstream quality control of taxa.

To estimate the taxonomic relative abundances at the species level for each sample, Bracken[8] is applied to the Kraken2 classification report files to compute the estimated species abundances using a Bayesian algorithm. To restimate the species abundances with Bracken, a kmer distribution file is generated to provide the expected number of kmers classified for each genome in the Kraken2 database. This is performed using bracken-build function with the Kraken2 database files (--db), kmer length (-k), and read length of the data (-l) provided as input:

```
bracken-build -d $DBNAME -t $NCORES -k $KMER_LEN -l $READ_LEN
```

Bracken v3.0.1 is run on the Kraken2 report of each sample ($SAMPLE.aggregated.report.txt) with input arguments kmer disterbution file (-k), taxonomic classification level to estimate abundances (-l), threshold of reads to remove low abundance taxa (-t).

```
python ${Bracken}/est_abundance.py -i ${SAMPLE}.aggregated.report.txt \
      -k ${DBNAME}/database${READ_LEN}mers.kmer_distrib \
      -l ${CLASSIFICATION_LVL} \
      -t ${THRESHOLD} \
      -o ${SAMPLE}.bracken.species.txt \
      --out-report ${SAMPLE}.bracken.report
```

The outputs of this step includes:
1. Bracken output file for each sample ($SAMPLEARG.bracken.species.txt) which provides the identified taxa classified for the read sequences with the number of additional reads mapped to each taxon and the total number of read counts and relative abundances assigned based on the additional reads.
2. Kraken-style Bracken report file ($SAMPLE.bracken.report) which identifies the taxa classified for the read sequences and the number of reads mapped to each taxon.

Scripts from KrakenTools[7] are implemented to merge the Bracken output files across all samples into a txt file for downstream analyses (merged.bracken.species.txt).

```
python ${BRACKEN}/combine_bracken_outputs.py --files ${BRACKEN_FILES} \
      --output merged.bracken.species.txt
```

From the taxonomic NCBI IDs in the combined bracken output file, we generate a taxonomic table of taxa names with each level of the taxonomy (e.g., kingdom to species) as columns and taxa as rows using the R package taxonomizr (https://github.com/sherrillmix/taxonomizr/).

Marker-gene-based taxonomic classification:

MetaPhlAn4[9] is a tool for taxonomic characterization of shotgun metagenomic sequencing data using a curated marker gene database (vJan25) . Paired-end fastq input files, the output from KneadData, are provided as input to run with MetaPhlAn4 v.4.2.2. The following options are used:

```
metaphlan  ${SAMPLE}_kneaddata_paired_1.fastq.gz,
${SAMPLE}_kneaddata_paired_2.fastq.gz
      --nproc 16 \
      --db_dir $DB_DIR \
      --input_type fastq \
      -o $OUTPUT_FILE \
      --mapout $TMPDIR/${SAMPLE}.bowtie2.bz2
```

The output of this step is one file per sample with relative abundance of taxa that sums to 1 at each level of taxonomy (e.g. species-genome-bin (SGB), genus, family). These outputs are merged into a single file that represents samples as columns and taxa as rows using a utility script within MetaPhlAn4 we adjusted to keep NCBI IDs (merge_metaphlan_tables_ncbi-included.py.py) and then stratified by taxonomic level (metaphlan4_clean_split_table.py). We combined these steps into one shell script (03_clean_merge_metaphlan.sh).

References

1 Beghini, F. *et al.* Integrating taxonomic, functional, and strain-level profiling of diverse microbial communities with bioBakery 3. *eLife* **10**, doi:10.7554/elife.65088 (2021).

2 Bolger, A. M., Lohse, M. & Usadel, B. Trimmomatic: a flexible trimmer for Illumina sequence data. *Bioinformatics* **30**, 2114-2120 (2014).

3 Benson, G. Tandem repeats finder: a program to analyze DNA sequences. *Nucleic acids research* **27**, 573-580 (1999).

4 Langmead, B. & Salzberg, S. L. Fast gapped-read alignment with Bowtie 2. *Nature methods* **9**, 357-359 (2012).

5 Ewels, P., Magnusson, M., Lundin, S. & Käller, M. MultiQC: summarize analysis results for multiple tools and samples in a single report. *Bioinformatics* **32**, doi:10.1093/bioinformatics/btw354 (2016/10/01).

6 Wood, D. E., Lu, J. & Langmead, B. Improved metagenomic analysis with Kraken 2. *Genome Biol* **20**, 257, doi:10.1186/s13059-019-1891-0 (2019).

7 Lu, J. *et al.* Metagenome analysis using the Kraken software suite. *Nat Protoc* **17**, 2815-2839, doi:10.1038/s41596-022-00738-y (2022).

8 Lu, J., Breitwieser, F. P., Thielen, P. & Salzberg, S. L. Bracken: estimating species abundance in metagenomics data. *PeerJ Computer Science* **3**, e104, doi:10.7717/peerj-cs.104 (2017).

9 Blanco-Míguez, A. *et al.* Extending and improving metagenomic taxonomic profiling with uncharacterized species using MetaPhlAn 4. *Nature Biotechnology*, 1-12 (2023).